CS0054
(INTELLIGENT SYSTEMS)

EXERCISE

4

NATURAL LANGUAGE PROCESSING (NLP)

Group Name:	**Jose Miguel Viado**
Section:	**AN42**
Professor:	**Ms. Crisola Tan**

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: E]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design Intelligent system based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Know and understand natural language processing & language technology.
- Familiarize with the approaches to NLP.
- Learn sample applications of NLP.

IV. BACKGROUND INFORMATION

What is NLP?

Working in Data Science and having a background in Technical Writing, I was drawn to the field of Natural Language Processing (NLP). Machines understanding language fascinates me, and I often ponder which algorithms Aristotle would have used to build a rhetorical analysis machine if he had the chance. If you're new to Data Science, getting into NLP can seem complicated, especially since there have been so many recent advancements in the field. It is hard to know where to start.

What can Machines Understand?

While a computer can be actually quite good at finding patterns and summarizing documents, it must transform words into numbers before making sense of them. This transformation is needed because machines "learn" thanks to mathematics, and math doesn't work very well on words. Before transforming the words into numbers, they are often cleaned of things like special characters and punctuation, and modified into forms that make them more uniform and interpretable.

What are Word Clouds?

Word clouds (also known as text clouds or tag clouds) work in a simple way: the more a specific word appears in a source of textual data (such as a speech, blog post, or database), the bigger and bolder it appears in the word cloud.

A word cloud is a collection, or cluster, of words depicted in different sizes. The bigger and bolder the word appears, the more often it's mentioned within a given text and the more important it is.

Also known as tag clouds or text clouds, these are ideal ways to pull out the most pertinent parts of textual data, from blog posts to databases. They can also help business users compare two different pieces of text to find the wording similarities between the two.

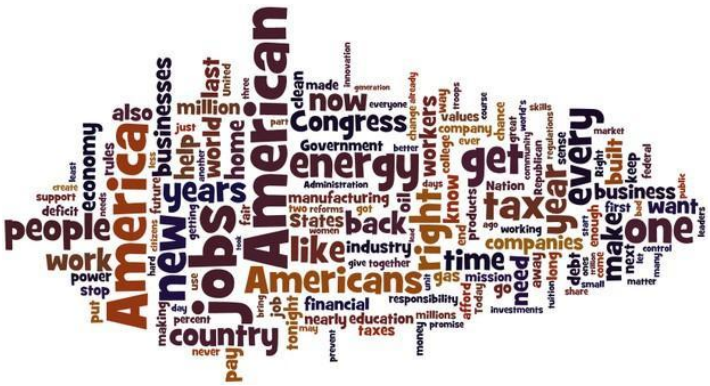
For instance, you might ask current customers what they like or don't like about your new product line. Or, you could ask them to give suggestions on how your organization could improve. They could also have the chance to elaborate on any pain points they're experiencing.

There are industry tools that allow you to code such open-ended data so users can understand it quantitatively. Yet, these don't come cheap. Word clouds offer a cost-effective, yet powerful, alternative.

With these, you can still quantify your text-based insights into measurable analytics. The only difference? You won't create a chart or graph as you would with a set of numbers.

Instead, you'll create a word cloud generator to transform the most critical information into a word cloud.

Here's an example from USA Today using U.S. President Barack Obama's State of the Union Speech 2012:



As you can see, words like "American," "jobs," "energy" and "every" stand out since they were used more frequently in the original text.

Now, compare that to the 2014 State of the Union address:



V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY (50 POINTS)

Data Set:

- **Spam Email Data Set found here:**
  https://github.com/bendgame/nlpBeginnerProjects/blob/master/NLP_beginner_projects.ipynb

Steps:

1. Importing Dependencies and Data

Start by importing the dependencies and the data. The data is stored as a comma separated values (csv) file, so use pandas' read_csv() function to open it into a DataFrame.

```
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

#create dataframe from csv
df = pd.read_csv('emails.csv')

df.head()
```

	text	spam
0	Subject: naturally irresistible your corporate...	1
1	Subject: the stock trading gunslinger fanny i...	1
2	Subject: unbelievable new homes made easy im ...	1
3	Subject: 4 color printing special request add...	1
4	Subject: do not have money , get software cds ...	1

df.head()

2. Exploratory Analysis

Before anything, it is best to do a quick analysis of the data to eliminate duplicate rows and establish some baseline counts. Use pandas drop_duplicates to drop the duplicate rows.

```
print("spam count: " +str(len(df.loc[df.spam==1])))
print("not spam count: " +str(len(df.loc[df.spam==0])))
print(df.shape)
df['spam'] = df['spam'].astype(int)

df = df.drop_duplicates()
df = df.reset_index(inplace = False)[['text','spam']]


print(df.shape)
```

```
spam count: 1368
not spam count: 4360
(5728, 2)
(5695, 2)
```

Counts and shape before/after duplicate removal

3. Cleaning the Texts

Notice in the word cloud image, all the text is lower case. There are no punctuation marks or special characters. That's because the text has been cleaned to make it easier to analyze. Using regular expressions, it is easy to clean the text using a loop:

```
clean_desc = []
for w in range(len(df.text)):
    desc = df['text'][w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("&lt;/?.*?&gt;"," &lt;&gt; ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)

#assign the cleaned descriptions to the data frame
df['text'] = clean_desc

df.head(3)
```

	text	spam
0	subject naturally irresistible your corporate ...	1
1	subject the stock trading gunslinger fanny is ...	1
2	subject unbelievable new homes made easy im wa...	1

4. Removing Stop Words

Stop words are the most common words like “the” and “of.” Removing them from the email text allows the more relevant frequent words to stand out. Removing stop words is a common technique! Some Python libraries like NLTK come pre-loaded with a list of stop words, but it is easy to create one from scratch.

```
stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or',
'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on',
're', 'new', 'subject']
```

5. Construct the Word Cloud

Conveniently, there is a Python library for creating word clouds. It can be installed using pip.

```
pip install wordcloud
```

When constructing the word cloud, it is possible to set several parameters like height and width, stop words, and max words. It is even possible to shape it instead of displaying the default rectangle.

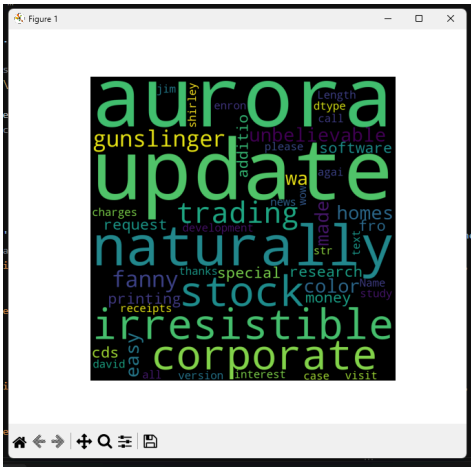
```
wordcloud = WordCloud(width = 800, height = 800, background_color =
'black', stopwords = stop_words, max_words = 1000
                      , min_font_size =
20).generate(str(df1['text']))

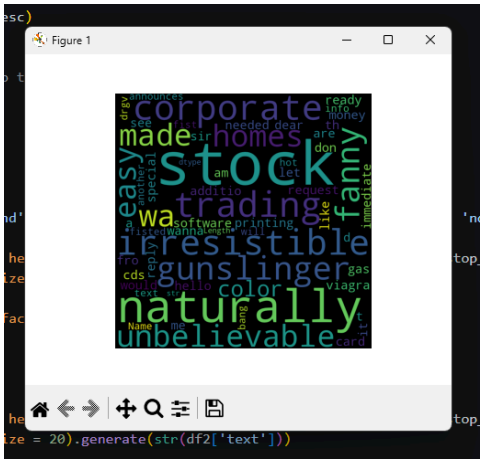

#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

The word cloud can be saved and displayed using matplotlib and .show(). This is the result of all records, regardless of it being spam.


corporate
cds
stock
receipts
development
text
call
aurora
ve
news
dtype
wow
interest
david
visit
than
color
charges
study
irresistible
jim
money
fanny
gunslinger
printing
enron
request
add
update
object
special
made
trading
homes
software
research
please
naturally
Name
case
unbelievable
easy
Length

OUTPUT:


Figure 1
aurora
update
naturally
irresistible
corporate
stock
gunslinger
unbelievable
trading
homes
fanny
research
color
money
printing
receipts
cds
easy
made
special
software
request
charges
wa


Figure 1
corporate
made
homes
stock
easy
trading
fanny
wa
irresistible
gunslinger
color
naturally
unbelievable
cds
software
printing
viagra
ize = 20).generate(str(df2['text']))

CODE:

```
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

df = pd.read_csv('emails.csv')
df.head()

print("spam count: " +str(len(df.loc[df.spam==1])))
print("not spam count: " +str(len(df.loc[df.spam==0])))
print(df.shape)
df['spam'] = df['spam'].astype(int)

df = df.drop_duplicates()
print(df.shape)

df = df.reset_index(inplace = False)[['text','spam']]

print(df.shape)
df['spam'].unique()

df.head()
clean_desc = []
for w in range(len(df.text)):
    desc = df['text'][w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("&lt;/?.*?&gt;"," &lt;&gt; ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)
#assign the cleaned descriptions to the data frame
df['text'] = clean_desc
df = df.reset_index()
df.head(3)

df1 =df.loc[df.spam==0]
df2 =df.loc[df.spam==1]
```

```
stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or',
'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on',
're', 'new', 'subject']
#set the word cloud parameters
wordcloud = WordCloud(width = 800, height = 800, background_color =
'black', stopwords = stop_words, max_words = 1000
                      , min_font_size =
20).generate(str(df['text']))
#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()

wordcloud = WordCloud(width = 800, height = 800, background_color =
'black', stopwords = stop_words, max_words = 1000
                      , min_font_size =
20).generate(str(df2['text']))
#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

CODE SCREENSHOT:

TA7_Viado.py U | TA9_Viado.py U | emails.csv U | TA5_Viado.py

Technical-Assessment-9 > TA9_Viado.py > ...

```python
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

df = pd.read_csv('emails.csv')
df.head()

print("spam count: " +str(len(df.loc[df.spam==1])))
print("not spam count: " +str(len(df.loc[df.spam==0])))
print(df.shape)
df['spam'] = df['spam'].astype(int)

df = df.drop_duplicates()
print(df.shape)

df = df.reset_index(inplace = False)[['text','spam']]

print(df.shape)
df['spam'].unique()

df.head()
clean_desc = []
for w in range(len(df.text)):
    desc = df['text'][w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("&lt;/?.*?&gt;"," &lt;&gt; ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)
#assign the cleaned descriptions to the data frame
df['text'] = clean_desc
df = df.reset_index()
df.head(3)
```

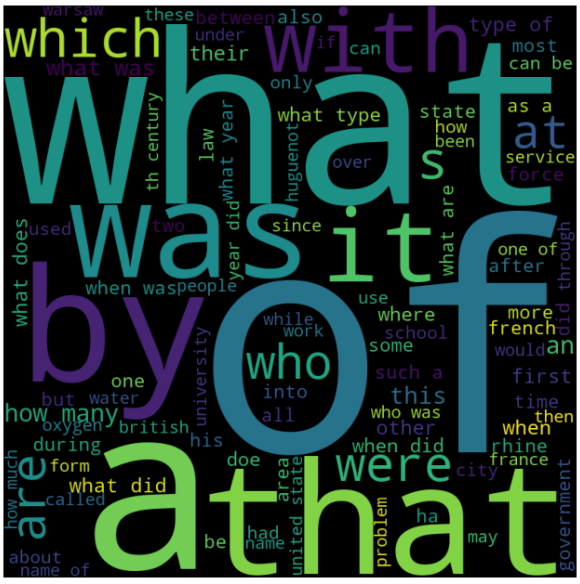
```python
df1 =df.loc[df.spam==0]
df2 =df.loc[df.spam==1]
stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or', 'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on', 're', 'new', 'subject']
#set the word cloud parameters
wordcloud = WordCloud(width = 800, height = 800, background_color = 'black', stopwords = stop_words, max_words = 1000
                      , min_font_size = 20).generate(str(df['text']))
#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()

wordcloud = WordCloud(width = 800, height = 800, background_color = 'black', stopwords = stop_words, max_words = 1000
                      , min_font_size = 20).generate(str(df2['text']))
#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

VII. CHALLENGE

- Repeat the procedure you did in Part VI., this time using any of the following open data sets found in this URL: https://analyticsindiamag.com/10-nlp-open-source-datasets-to-start-your-first-nlp-project/
- Clearly specify the intention of the output that you will generate.
- Compare and contrast sample outputs that you will generate.
- Provide a conclusion on whether you have achieved your goal or not.
- Provide a recommendation on things you did wrong and suggestions to improve and achieve the expected output.
- You may use existing Word Cloud generator like Wordle, Tagxedo, Tag Crowd, WordItOut, Word Cloud Generator for Chrome, Word Cloud Python tools or Google Word Cloud tools and check your output against the output of the generators. Analyze and explain results.

OUTPUT:



PYTHON GENERATED **WORDCLOUDS.COM GENERATED**

CODE:

```
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

df = pd.read_csv('squads.csv')

df.head()
```

```
print(df.shape)

text_cols = [c for c in df.columns if c.endswith('/question') or
c.endswith('/context')]
text_list = df[text_cols].stack().dropna().astype(str).tolist()
print(len(text_list))

clean_desc = []
for w in range(len(text_list)):
    desc = text_list[w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("</?.*?>"," <> ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

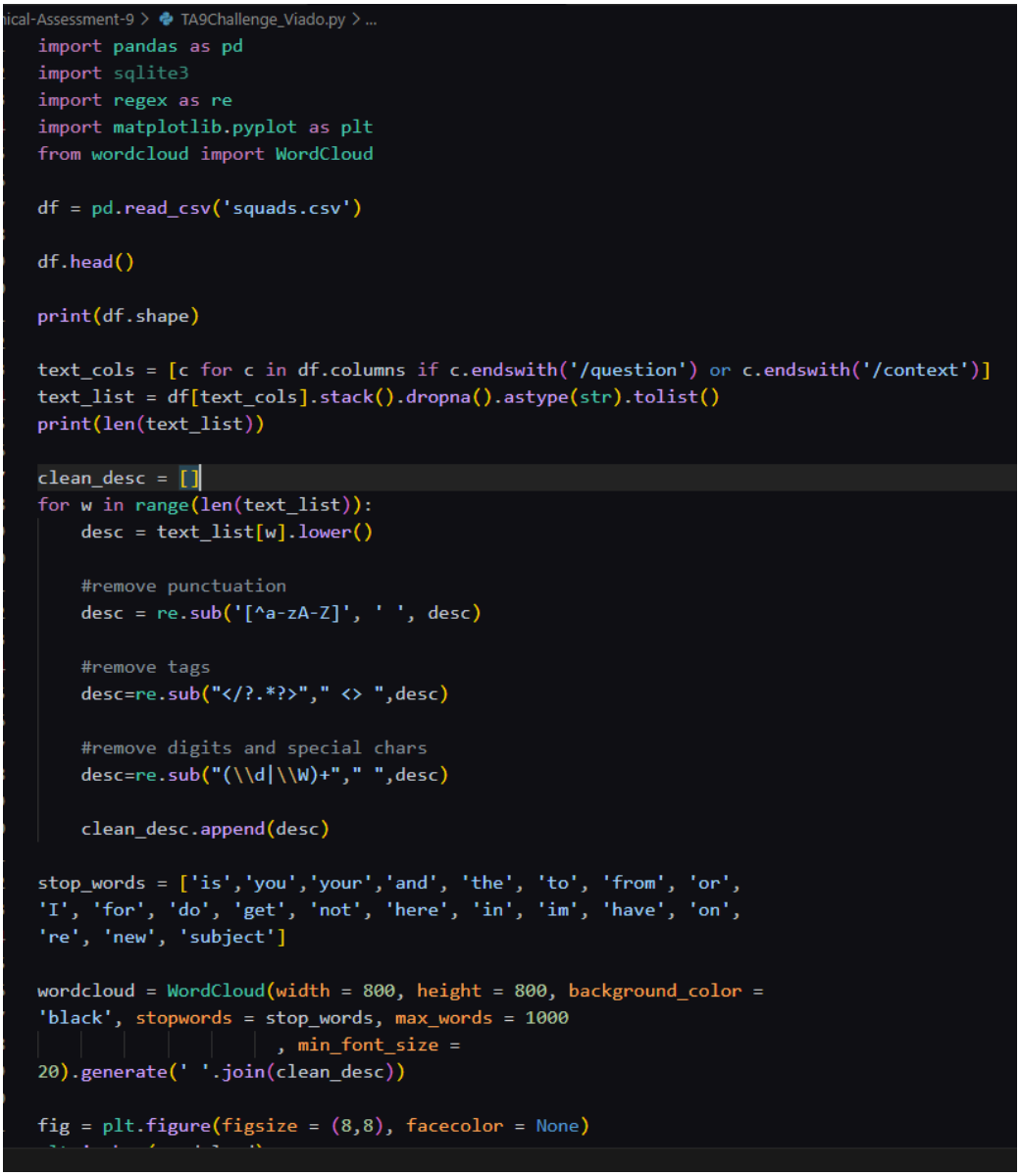
    clean_desc.append(desc)

stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or',
'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on',
're', 'new', 'subject']

wordcloud = WordCloud(width = 800, height = 800, background_color =
'black', stopwords = stop_words, max_words = 1000
                      , min_font_size =
20).generate(' '.join(clean_desc))

fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.savefig('cloud_text_csv.png')
plt.show()
```

CODE SCREENSHOT:



```
ical-Assessment-9 > TA9Challenge_Viado.py > ...
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

df = pd.read_csv('squads.csv')

df.head()

print(df.shape)

text_cols = [c for c in df.columns if c.endswith('/question') or c.endswith('/context')]
text_list = df[text_cols].stack().dropna().astype(str).tolist()
print(len(text_list))

clean_desc = []
for w in range(len(text_list)):
    desc = text_list[w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("</?.*?>"," <> ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)

stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or',
'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on',
're', 'new', 'subject']

wordcloud = WordCloud(width = 800, height = 800, background_color =
'black', stopwords = stop_words, max_words = 1000
                      , min_font_size =
20).generate(' '.join(clean_desc))

fig = plt.figure(figsize = (8,8), facecolor = None)
```

I tried generating the word cloud in Python and removing the stop words, so i could say my goal is achieved. I generated and compared the word clouds, which showed common question words like what, how many, and which. The word cloud python generated is cleaner than the website but the improvement is minimal.

VIII. REFERENCES

- Vahid Mirjalili(2017). Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow. India: Packt Publishing
- Numerical Algorithms: Methods for Computer Vision, Machine Learning, and Graphics

Online Sources:

- https://boostlabs.com/blog/what-are-word-clouds-value-simple-visualizations/
- https://github.com/bendgame/nlpBeginnerProjects/blob/master/NLP_beginner_projects.ipynb
- https://towardsdatascience.com/3-super-simple-projects-to-learn-natural-language-processing-using-python-8ef74c757cd9